

Cluster access tutorial

To access the cluster and open the links provided in the tutorial you need to be connected to the EPFL wifi or [EPFL VPN](#). Links in the tutorial are provided for you to be able to refer to the original sources if you want more details about the steps or if something is not working. However, you should be able to set up the job submission by simply following the instructions in the tutorial. Every new `bash` command in the tutorial is separated by an empty line (if no empty line is present the command is continued to the next line).

The code shown in `this font` is meant to be run in a terminal. If the code appears across multiple lines without any blank lines in between, treat it as a **single command**. A **blank line** between code blocks means you should run the next part as a **separate command**. All *sources in grey italics* are available as a reference, in case you want to understand the concepts better, but they are not essential for completing this tutorial.

Links to the documentation, where you can read information additional to the tutorial

RCP Wiki: <https://wiki.rcp.epfl.ch/en/home/CaaS>

Run:ai: <https://docs.run.ai/v2.18/Researcher/cli-reference/Introduction/>

Getting familiar with shell tools: <https://missing.csail.mit.edu/>

Login to RunAI

RCP documentation: <https://wiki.rcp.epfl.ch/home/CaaS/FAQ/how-to-use-runai>

Steps in the following section have to be done only once - the first time you connect to the cluster. You don't need to repeat them every time you submit a job. However, RunAI authentication expires after some time of inactivity and your RunAI commands might stop working, in such case simply repeat the RunAI login steps.

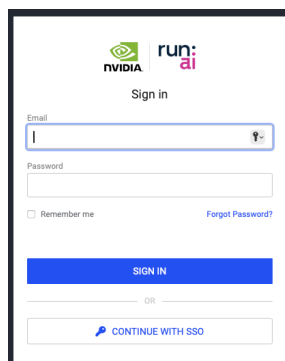
- 1) SSH to the jumphost. To do this, open terminal, and execute the following in the command line:

```
ssh <username>@jumphost.rcp.epfl.ch
```

Replace `<username>` with your gaspar username, and enter your password.

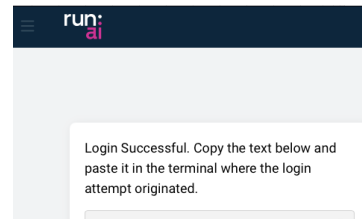
As a result, you should see something like this: `<username>@haas001:~$`

- 2) Login to the RunAI system.
 - a) On the server run the command:
`runai login`
 - b) Follow the link outputted by the previous command in the terminal and go to the website. (If asked to log in, press "Continue with SSO")



- c) Copy the text that appears in the special window and insert it into the command line after the `Enter` verification code:

```
@haas001:~$ runai login
Go to the following link in your browser:
Copy this link to the → https://app.run.ai/auth/...
browser to get a token
[Enter verification code: ← Insert your token here
INFO[0071] Logged in successfully
```



- 3) Run the next command:

```
runai config project course-ee-559-${USER}
```

It sets the course project as the default project on rcp. In this command keep `${USER}` as it is, since this is the system variable, and it will be pulled automatically.

Now you can run jobs!

How to run a job

RCP documentation: <https://wiki.rcp.epfl.ch/home/CaaS/FAQ/how-to-submit-job-demo>

- 1) From your local computer copy the project files to the home directory on the server (jumphost). To do that in the terminal on your local computer, go to the directory where the `practice_3_repository` is located and run the following command:

```
scp -r practice_3_repository <username>@jumphost.rcp.epfl.ch:/home/<username>/
```

Replace `<username>` with your Gaspar username.

The command `scp -r` copies the directory from your local pc to the server. Simply it works like

```
scp      -r      <path-to-the-directory-on-your-local-pc>
<server-address>:<path-to-the-directory-on-server>
```

You can familiarise yourself with this and other bash commands here <https://missing.csail.mit.edu/> and read more about `scp -r` here <https://www.geeksforgeeks.org/scp-command-in-linux-with-examples/>.

Similarly, go to the directory where `fruit_dataset` is located on your local computer and copy it to the home directory on the server too:

```
scp -r fruit_dataset <username>@jumphost.rcp.epfl.ch:/home/<username>/
```

- 2) Find out your `LDAP_USERNAME` / `UID` / `GROUPNAME` etc. To do that ssh to the jumphost as before

```
ssh <username>@jumphost.rcp.epfl.ch
```

and run the command:

```
id
```

Example:

```
$ ssh mfontes@jumphost.rcp.epfl.ch

mfontes@haas001:~$ id
uid=248658(mfontes) gid=11202(RCP-GE-StaffU) groups=11202(RCP-GE-StaffU),20249(VPA-AVP-CP-StaffU), ...
```

Save the printed parameters somewhere for yourself to use later and exit the ssh connection.
We will further refer to your id as <Your-UID>.

- 3) After connecting to the jumphost (where you reached after `ssh <username>@jumphost.rcp.epfl.ch`) submit the job through the runAI.

To submit a job, you need to execute the command `runai submit <parameters>`, where <parameters> should include the details on the type of job you want to run.

For the parameters, you must use the following ones:

`--run-as-uid <Your-UID>`: Your uid that you have found using “id” command.

`--existing-pvc claimname=<name>,path=<path>`: Mounts an existing course-wide volume to a specific path, it allows your job to attach the folder <name> at the path <path>.

Directories available under `--existing-pvc` on the cluster:

When you run a job with `--existing-pvc claimname=<name>` the folder named <name> that is available on cluster at ‘Actual path’ will be connected to your job and will be accessible under a new path (‘Path inside a job’, <path>), therefore to access the files that were initially saved at ‘Actual path’ inside the job your would need to go to the ‘Path inside a job’.

Actual path	<name>	Path inside a job	Note
/home/<username>	home	/home/<username>	Your personal space.
/mnt/course-ee-559/rcp-caas-ee-559-g<Your-Group-number>/scratch-g<Your-Group-number>	course-ee-559-scratch-g<Your-Group-number>	/scratch	Your shared group directory <Your-Group-number> is the group number you registered on Moodle; it should contain two symbols, e.g. 01, 15, 27.
/mnt/course-ee-559/shared-ro	course-ee-559-shared-ro	/shared-ro	The folder where we can put large models and datasets by request, and you will be able to only read them.
/mnt/course-ee-559/shared-rw	course-ee-559-shared-rw	/shared-rw	The folder where you can put large models and datasets, however, everyone from the course can see it!

`--image <docker_image_name>`: Specifies which Docker image should be used for the job.

`--command -- <command_to_run>`: Specifies the command to execute. To run a Python file, use `python3 file.py`.

`--gpu <number>`: Sets the number of GPUs needed. You can request a fraction of a GPU (e.g. `--gpu 0.2`). Do not request more than 1 unless your code specifically requires multi-GPU training.

If you want to use a specific GPU, you can use an additional parameter `--node-pools`

`<names_of_gpus_you_want>`: The parameter allows you to exactly specify the types of GPUs you want to use (e.g. `--node-pools v100`). You have access to V100, A100-40G and A100-80G GPUs. Please, be mindful of the resources you use.

To submit a job run:

```
runai submit --run-as-uid <Your-UID> --image
registry.rcp.epfl.ch/ee559/environment-with-packages:latest --gpu 1 --existing-pvc
claimname=course-ee-559-scratch-g<Your-Group-number>,path=/scratch
--existing-pvc claimname=home,path=/home/<username> --existing-pvc
claimname=course-ee-559-shared-ro,path=/shared-ro --existing-pvc
claimname=course-ee-559-shared-rw,path=/shared-rw --command -- python3
/home/<username>/practice_3_repository/practice_3_simplified.py --dataset_path
/home/<username>/ --results_path /home/<username>/practice_3_repository/results/
```

Note: the `--existing-pvc` parameter for a folder is not necessary if you don't need to use the content inside the folders.

The `practice_3_repository/practice_3_simplified.py` script contains training of the model that was provided in Practice 3 and saving of the results. When your job is complete, the trained model checkpoint will appear in the `/home/<username>/practice_3_repository/results/checkpoint/` directory and the .png graphs of the losses will appear in the `/home/<username>/practice_3_repository/results/` directory.

To use interactive mode (time limit: 12h for each session, a session will be killed if no GPU is used within 2 consecutive hours) run:

```
runai submit --run-as-uid <Your-UID> --image
registry.rcp.epfl.ch/ee559/environment-with-packages:latest --gpu 1 --existing-pvc
claimname=course-ee-559-scratch-g<Your-Group-number>,path=/scratch
--existing-pvc claimname=home,path=/home/<username> --existing-pvc
claimname=course-ee-559-shared-ro,path=/shared-ro --existing-pvc
claimname=course-ee-559-shared-rw,path=/shared-rw --interactive --attach
```

Note: make sure to delete an interactive job after you are done using it

```
runai delete job <job-name> -p course-ee-559-<username>
```

4) Other commands to interact with runAI:

- a) Output the logs for the job. Logs contain information that was printed when running the job or errors if the job failed:

```
runai logs <job-name>
```

- b) Delete the job:

```
runai delete job <job-name> -p course-ee-559-<username>
```

- c) List all jobs:

```
runai list
```

d) Describe the status of the job:

```
runai describe job <job-name> -p course-ee-559-<username>
```

How to build your own Docker image

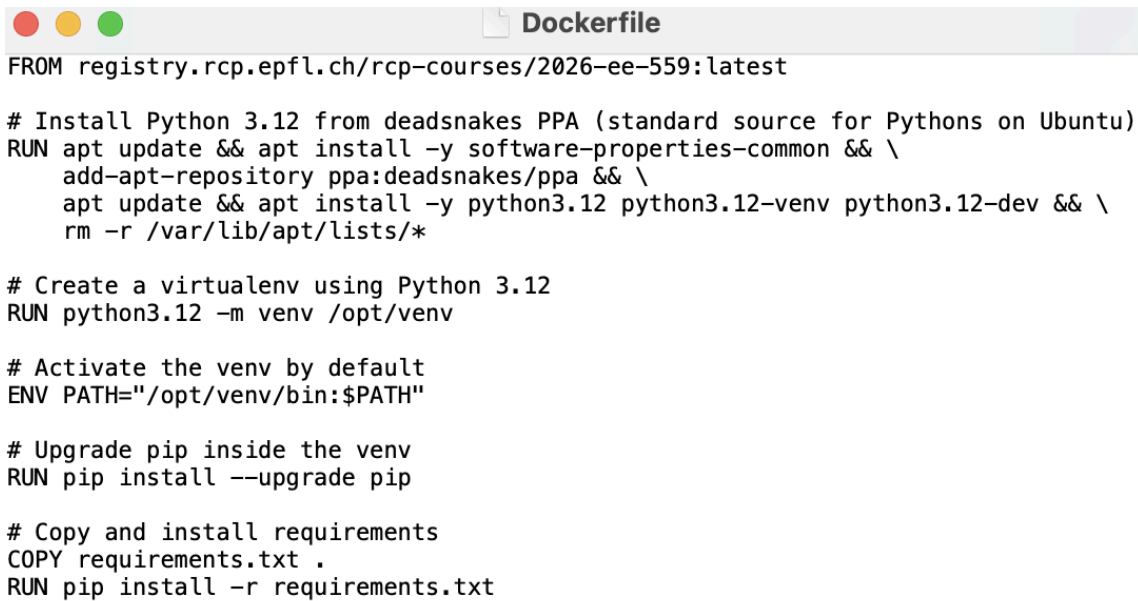
If you need to use additional libraries that are not included in the Docker image provided for you (registry.rcp.epfl.ch/ee559/environment-with-packages:latest), you would need to create your own image.

Please follow the instructions below to do so.

<https://wiki.rcp.epfl.ch/home/CaaS/FAQ/how-to-build-a-container-part1>

<https://wiki.rcp.epfl.ch/en/home/CaaS/FAQ/how-to-build-a-container-part2>

- 1) **On your local computer**, install Docker. You can download it here <https://www.docker.com/>
- 2) You would need to create a Docker image on your **local computer**, and for that create a folder where you are going to do that. Create a folder `ee559_docker_env`. In this folder, create a `requirements.txt` file, where you will list all the necessary requirements to run your project, and a `Dockerfile`, which will contain all of the instructions for building a Docker image.
- 3) Prepare the Dockerfile for your Docker image:
 - a) Take the Dockerfile we prepared for you. Carefully read the file. Snapshot of the file:



```
FROM registry.rcp.epfl.ch/rcp-courses/2026-ee-559:latest

# Install Python 3.12 from deadsnakes PPA (standard source for Pythons on Ubuntu)
RUN apt update && apt install -y software-properties-common && \
    add-apt-repository ppa:deadsnakes/ppa && \
    apt update && apt install -y python3.12 python3.12-venv python3.12-dev && \
    rm -r /var/lib/apt/lists/*

# Create a virtualenv using Python 3.12
RUN python3.12 -m venv /opt/venv

# Activate the venv by default
ENV PATH="/opt/venv/bin:$PATH"

# Upgrade pip inside the venv
RUN pip install --upgrade pip

# Copy and install requirements
COPY requirements.txt .
RUN pip install -r requirements.txt
```



Security:

- Please do not hardcode any credential/passwords/API keys or any other secret in the Dockerfile.

- Use caution when pulling public container images. As you can imagine, the public container registry has a variety of images and some of these may contain malicious code. Pay special attention to things like: image name typos, the number of releases and image update cycle.

- b) In `Dockerfile` the base image from which you can create your own image is `registry.rcp.epfl.ch/rcp-courses/2026-ee-559:latest`. This base image allows you to share the image you build on top across the team members without the need for each student to create their own. If as a group you want to use the same environment, it is enough for only one group member to create an image and upload it to the registry.
 - c) If you want to change the python version from 3.12 to older versions, for example 3.10, you need to replace 3.12 with 3.10 everywhere in the Dockerfile. Python versions starting from 3.7 should be supported.

- 4) In your `requirements.txt` file write down the Python libraries that you may need for your project and that are **not included** in the base image on which you are creating your docker.

```
1 torch==2.4.1 --index-url https://download.pytorch.org/whl/cu121
2 torchvision==0.19.1 --index-url https://download.pytorch.org/whl/cu121
3 matplotlib
4 numpy
5 pandas
6 tqdm
7 scikit-learn
```

- 5) On your local computer, build docker image: `ee559-mini-project-test`, in terminal go to the `ee559_docker_env` folder and run the following command, changing the `<username>` everywhere to your Gaspar username:

```
docker build --platform linux/amd64 . --tag
registry.rcp.epfl.ch/ee-559-<username>/my-toolbox:v0.1
```

The `--tag` parameter `:v0.1` defines the version of your Docker image. `registry.rcp.epfl.ch` is needed so that later you can push your Docker image to the registry. If later in the project you would need to update the packages, you can rebuild your Docker while preserving the name of the project and only changing the version, e.g. to 0.2. `LDAP_GROUPNAME` and `LDAP_GID` are fixed for the course and should not be changed.

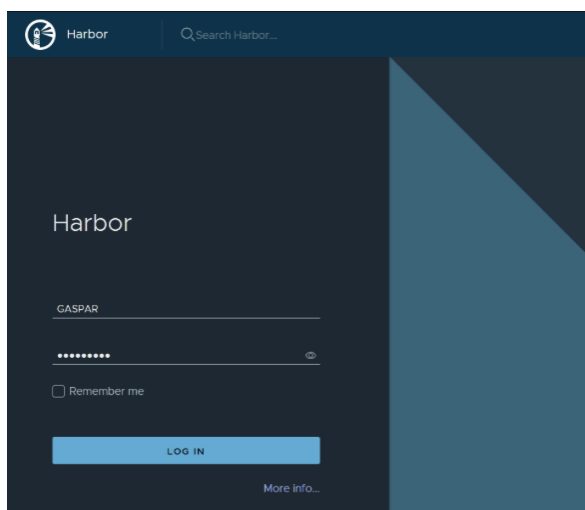
- 6) Check that the image is already built:

```
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
registry.rcp.epfl.ch/ee-559-<username>/my-toolbox	v0.1	127cc75acea5	13 minutes ago	18GB

- 7) Transfer the image to the registry so that you will be able to have access to it on the cluster <https://wiki.rcp.epfl.ch/home/CaaS/FAQ/how-to-registry>:

- a) Login to <https://registry.rcp.epfl.ch> with your gaspar username and password



- b) Press New project and create a new **PUBLIC** project with the name `ee-559-<username>`. Replace `<username>` with your Gaspar username. Same as the one you previously built a Docker with.

New Project

Project Name *

Access Level (i) ☒ Public

- c) Go back to the command line of your PC and log in to the registry with Docker. Insert your Gaspar username and password.

```
docker login registry.rcp.epfl.ch
```

d)

```
Username: <username>
```

i Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit <https://app.docker.com/settings>

```
Password:
Login Succeeded
```

- e) Push the Docker image to the registry with the command

```
docker push registry.rcp.epfl.ch/ee-559-<username>/my-toolbox:v0.1
```

- 8) To run the job with your custom container
registry.rcp.epfl.ch/ee-559-username/my-toolbox:v0.1. To do that in runai submit use the parameter `--image registry.rcp.epfl.ch/ee-559-<username>/my-toolbox:v0.1`
- 9) If later you want to update your image with new Python libraries, you should append the new libraries to the end of requirements.txt and repeat steps 5, 6, and 7c-d with updating the version of the image (to v0.2, for example).

Success!

Summary: recipe for running your project

- 1) Create your own Docker image or find a base image that contains all of the necessary libraries.
- 2) Copy project files and dataset to your home directory (`/home/<username>/`) and shared group directory `/mnt/course-ee-559/rcp-caas-ee-559-g<Your-Group-number>/scratch/`, where `<Your-Group-number>` should be replaced with the number of your group (01-50) on the server.

For the project files, you can use github repository to store all the code and collaborate. You can find

information about how to create a github repo here <https://docs.github.com/en/get-started/start-your-journey/hello-world> . Make sure to make your repository is **private**. Then you can just clone your repository on the server with `git clone repository_name` and pull updated versions of the code with `git pull`.

Storing the project in a shared directory is needed so that you can easily exchange files on the server (e.g. if someone from the group already trained the model, you can share the weights with your collaborators on the shared directory of your group). Here we have shown how to work with the project taking as an example `practice_3_repository`, which contains multiple Python files that interact with each other.

If you have never used `scp` command in a command line before, you can read more about it here <https://www.geeksforgeeks.org/scp-command-in-linux-with-examples/> .

To copy these files to the home directory on the server, on your local pc go to the folder where `practice_3_repository` and `fruit_dataset` folders are located and run the `scp -r <path-to-the-directory-on-your-local-pc> <server-address>:<path-to-the-home-directory-on-server>` command:

```
scp -r practice_3_repository <username>@jumphost.rcp.epfl.ch:/home/<username>/
```

```
scp -r fruit_dataset <username>@jumphost.rcp.epfl.ch:/home/<username>/
```

To copy these files to the shared directory on the server:

```
scp -r fruit_dataset  
<username>@jumphost.rcp.epfl.ch:/mnt/course-ee-559/rcp-caas-ee-559-g<Your-Group-number>/scratch/
```

- 3) Test your script first on an interactive node. Jump to an interactive node specifying the exact Docker image that you want to use during the training.

```
runai submit --run-as-uid <Your-UID> --image  
registry.rcp.epfl.ch/ee-559-<username>/my-toolbox:v0.1 --gpu 1 --existing-pvc  
claimname=course-ee-559-scratch-g<Your-Group-number>,path=/scratch  
--existing-pvc claimname=home,path=/home/<username> --existing-pvc  
claimname=course-ee-559-shared-ro,path=/shared-ro --existing-pvc  
claimname=course-ee-559-shared-rw,path=/shared-rw --interactive --attach
```

Run the Python script on the interactive node to test that your file is running correctly (do it on fewer epochs, do not do the full training on the interactive node, since this way you are going to occupy resources).

```
python3 /home/<username>/practice_3_repository/practice_3_simplified.py  
--dataset_path /home/<username>/ --results_path  
/home/<username>/practice_3_repository/results/
```

Check that you can also run the jobs from the group directory. To do that first copy the materials from your home to the group directory and run the following command:

```
python3 /scratch/practice_3_repository/practice_3_simplified.py --dataset_path  
/scratch/ --results_path /scratch/practice_3_repository/results/
```

- 4) Submit the job:

```
runai submit --run-as-uid <Your-UID> --image
```



```
registry.rcp.epfl.ch/ee559/environment-with-packages:latest --gpu 1 --existing-pvc
claimname=course-ee-559-scratch-g<Your-Group-number>,path=/scratch --existing-pvc
claimname=home,path=/home/<username> --existing-pvc
claimname=course-ee-559-shared-ro,path=/shared-ro --existing-pvc
claimname=course-ee-559-shared-rw,path=/shared-rw --command -- python3
/home/<username>/practice_3_repository/practice_3_simplified.py
--dataset_path /home/<username>/ --results_path
/home/<username>/practice_3_repository/results/
```

- 5) If your job failed you can check the logs with
`runai logs <job-name>` and follow the status here <https://rcpepfl.run.ai/>

Things to be careful about and additional info

- 1) Make sure that you exactly follow all the instructions! This process contains multiple steps and it is sometimes much harder to debug than to do things from scratch.
- 2) **Store checkpoints** of your models. When you are running the python file, all printed results will be deleted. You can reuse and adopt the file `practice_3_repository/checkpoint_utils.py` for your project where the storing of model weight and optimiser is implemented.
- 3) When updating your own docker image, make sure that installed python version and installed libraries are compatible with the version of the library that you are trying to add. E.g. if you are using the python 3.12 it will be incompatible with the torchaudio since it is based on python 3.12.3 and torchaudio can only be used with python <= 3.11 (compatibility label can be found here <https://pytorch.org/audio/main/installation.html>). Therefore, if you need torchaudio for your project you might want to create a new image with another python version or use other ready images that have torchaudio already installed.
- 4) If a lot of students occupied too many resources some of the jobs might be interrupted, if the server is going to be needed for other EPFL needs. That's again why it is important to save checkpoints of the model, optimiser, scheduler and intermediate results.
- 5) If you submit a training job requesting one or more GPUs but don't use them for two hours, the job is going to be automatically suspended to free up resources for other users in the cluster. This doesn't prevent you from submitting a new job. If you don't need GPUs, you can submit a job without requesting any. If you need a GPU for live testing, request an interactive job using the "--interactive" flag, as this type of job allows you to keep the GPU allocated without continuous usage.
- 6) Since you are going to work as a group and on different platforms, try making code easily adaptable to different platforms and repositories. It means **no hardcoding!** E.g. Specify
`device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`
Pass all the PATHs as argparse arguments (as in the `practice_3_repository/practice_3_simplified.py` file), and do not hardcode any paths, api keys and so on.
- 7) If you want more details about the command, you can always run `command --help`, e.g. `runai submit --help`, which will output some details about the parameters of the command
- 8) Error 'can't open file' or 'file not found' might signal that you specified the wrong path to the file. First, check that the path to the file is correct.
- 9) If you don't want to type the password every time you ssh to the server, you can [create an ssh key](#).